

Ce cours s'inspire du cours éponyme du master IMSD (Ingénierie Mathématique et Science des Données) de l'Université de Lorraine, que j'ai suivi à l'automne 2025 auprès du Dr. Wei Lu.

Chapitre 1

Introduction à la théorie de l'Apprentissage

Contexte pour les fameux datasets MNIST et IRIS:

- Représenter chaque object i par un vecteur $x_i \in X \subset \mathbb{R}^d$ avec X : espace des caractères (features)
- Objectif : définir une fonction f qui associe à chaque oervation x_i son label $f(x_i) \in Y$.
- Classification : $Y = \{1, ..., K\}$. La fonction f est appelé classifieur.
- Classification binaire $|Y| = 2$. Par exemple $Y = \{-1, 1\}$

Quand on a un ensemble de donnée, on ne cherche pas une fonction qui correspond exactement aux données, mais une fonction qui généralise bien, c'est à dire qui prédit bien les nouvelles données. Il faut chercher un compromis entre la *capacité de prédiction* du modèle et sa *complexité*.

Démarche

Le point de départ est un ensemble dit d'apprentissage pour lequel le label est connu. l'objectif est d'élaborer un modèle paramétrique puis d'estimer ses paramètres sur le *jeu de données d'apprentissage*. Ce modèle permettra de prendre des décisions pour de futures (nouvelles) données. Par ailleurs, on peut tester l'erreur du modèle sur un *jeu de données de test*, ce qui permet d'avoir *l'erreur de généralisation*. Si le modèle est trop simple, l'erreur d'apprentissage sera élevée.

Comment sélectionner le meilleur modèle ?

Il faut que l'erreur soit mesurable, pour cela on utilise le *jeu de données de test*. Une hypothèse importante est que la distribution des données d'apprentissage est représentative de celle des données futures. La question qu'on est légitime de se poser est : et-ce que minimiser l'erreur d'apprentissage permet de minimiser l'erreur de généralisation ? *Spoiler* : Non, cela peut causer du sur-apprentissage (overfitting).

Constats

Ce qu'on constate, c'est que le modèle qui a la plus faible erreur d'apprentissage n'est pas forcément celui qui a la plus faible erreur de test. L'erreur d'apprentissage est en général une estimation optimiste de l'erreur de test. L'écart entre erreur d'apprentissage et erreur de test est appelé *biais de sélection*. C'est à dire qu'elle dépend du modèle.

Questionnement

Si on ne peut pas mesurer l'erreur de généralisation, comment l'estimer ? *Spoiler* : En utilisant l'erreur sur les données de tests, non utilisées pour l'apprentissage. Grâce à une éventuelle *borne supérieure théorique* sur l'écart entre erreur d'apprentissage et erreur de généralisation :

- erreur de généralisation $\leq$ erreur d'apprentissage + borne

Lorsqu'elle existe, cette borne peut être trop élevée pour être exploitable.

Cadre mathématique

- Hypothèse : on dispose d'un ensemble D_n dit d'apprentissage, qui est constitué de n points $(X_i, Y_i) \in X \cdot Y$ qui sont des tirages aléatoires i.i.d de $(X, Y) \sim \mathbb{P}_{X,Y}$.
- On se donne $L : Y \cdot Y \rightarrow \mathbb{R}^+$, une fonction de perte, que l'on suppose bornée.

- Exemples :

- Perte 0 - 1 : Nulle si prédiction correcte, 1 sinon.
- Hinge loss : $L(y, y') = \max(0, 1 - yy')$
- Perte quadratique : $L(y, y') = (y - y')^2$

- Le classifieur f doit minimiser l'erreur moyenne pour une nouvelle observation ou erreur de généralisation :

$$R(f) = \mathbb{E}[L(Y, f(X))] = \int_{X \times Y} L(y, f(x)) d\mathbb{P}_{X,Y}(x, y)$$

et $f \in F$ avec F la classe de fonctions connue.

- Cette classe de fonctions F est éventuellement paramétrique.

Dans ce cas, on note $F = \{f_\theta, \theta \in \Theta\}$.

- Le problème est que l'on ne connaît pas la distribution $\mathbb{P}_{X,Y}$.

- On va remplacer l'intégrale par une forme explicite de l'erreur empirique :

$$\hat{R}(f, D_n) = \frac{1}{n} \cdot \sum_{i=1}^n L(f(X_i), Y_i)$$

- Principe **ERM (Empirical Risk Minimisation)** : remplacer le problème P par le problème suivant :

$$P_n : \text{trouver } f_n \in F \text{ t.q } \hat{R}(f_n, D_n) = \min_{f \in F} \hat{R}(f, D_n)$$

- En effet, on choisit $f_n = \operatorname{argmin}_{f \in F} \hat{R}(f, D_n)$, on prend le modèle qui minimise l'erreur sur les données qu'on a.

Rappel : convergence en probabilité

Soit $X, X_1, ..., X_n$, des variables aléatoires. On dit que X_n converge vers X en probabilité, ce qu'on note $X_n \xrightarrow{(p)} X$ si pour tout $\varepsilon > 0$, on a :

$$\lim_{n \rightarrow \infty} \mathbb{P}(|X_n - X| > \varepsilon) = 0$$

Consistance du principe ERM : Définition

Est-ce que le modèle qui marche bien sur l'échantillon marchera aussi bien en vrai sur la distribution ? C'est là qu'intervient la notion de consistance.

Le principe ERM est dit constant si les deux conditions suivantes sont vérifiées simultanément :

$$\hat{R}(f_n, D_n) - R(f_n) \xrightarrow{(p)} 0 \text{ et } \hat{R}(f_n, D_n) \xrightarrow{(p)} \inf_{g \in F} R(g)$$

Note : C'est à dire que l'erreur empirique et l'erreur réelle convergent vers la même valeur pour un grand nombre de données et que l'erreur du modèle choisi f_n converge vers l'erreur minimale possible.

Le principe ERM : Théorème de Vapnik 1981

Le principe ERM est constant ssi pour tout $\varepsilon > 0$, on a :

$$\lim_{n \rightarrow \infty} \mathbb{P}\left(\sup_{g \in F} |R(g) - \hat{R}(g, D_n)| > \varepsilon\right) \rightarrow 0$$

Les convergences sont en probabilité. C'est à dire que $\sup_{g \in F} |R(g) - \hat{R}(g, D_n)| \rightarrow 0$.

Preuve : Hypothèse clé : l'erreur empirique $\hat{R}(g, D_n)$ se rapproche uniformément de l'erreur réelle $R(g)$ pour tous les modèles $g \in F$.

Si pour tout $\varepsilon > 0$,

$$\lim_{n \rightarrow \infty} \mathbb{P}\left(\sup_{g \in F} |R(g) - \hat{R}(g, D_n)| > \varepsilon\right) \rightarrow 0$$

, montrons la consistance de l'ERM.

Comme $\sup_{g \in F} |R(g) - \hat{R}(g, D_n)| \rightarrow \varepsilon \rightarrow 0$ ainsi, il en va de même pour $f_n \in \mathbb{R}, |\hat{R}(f_n, D_n) - R(f_n)| \rightarrow 0$

Soit $h^* \in F$ tel que $R(h^*) = \inf_{g \in F} R(g)$. C'est le meilleur modèle théorique dans la classe F . Par définition de f_n , on a :

$$\hat{R}(f_n, D_n) \leq \hat{R}(h^*, D_n)$$

. En effet, f_n minimise l'erreur empirique et h^* minimise l'erreur réelle mais pas forcément l'erreur empirique.

On peut alors écrire :

$$R(f_n) - R(h^*) = \left(R(f_n) - \hat{R}(f_n, D_n)\right) + \left(\hat{R}(f_n, D_n) - \hat{R}(h^*, D_n)\right) + \left(\hat{R}(h^*, D_n) - R(h^*)\right)$$

Comme

$$\hat{R}(f_n, D_n) \leq \hat{R}(h^*, D_n)$$

il vient que :

$$\left(\hat{R}(f_n, D_n) - \hat{R}(h^*, D_n)\right) \leq 0 \text{ et}$$

$$R(f_n) - R(h^*) \leq \left(R(f_n) - \hat{R}(f_n, D_n)\right) + \hat{R}(h^*, D_n) - R(h^*)$$

Ainsi,

$$R(f_n) - \inf_{g \in F} R(g) \leq 2 \sup_{g \in F} |R(g) - \hat{R}(g, D_n)|$$

Donc

$$\mathbb{P}\left(|R(f_n) - \inf_{g \in F} R(g)| > \varepsilon\right) \leq \mathbb{P}\left(2 \sup_{g \in F} |R(g) - \hat{R}(g, D_n)| > \varepsilon\right)$$

Donc

$$\hat{R}(f_n, D_n) - R(f_n) \rightarrow 0 \text{ (point 1 de la consistance)}$$

De plus,

$$R(f_n) \rightarrow \inf_{(p)} R(g) \text{ (point 2 de la consistance)}$$

CQFD

Le principe ERM : Un cas particulier

Si la famille F est finie (nombre de fonctions fini), comme pout tout $g \in F$, par définition,

$$\hat{R}(g, D_n) = \frac{1}{n} \cdot \sum_{i=1}^n L(g(X_i), Y_i)$$

- Les V.A. $Z_i = L(g(X_i), Y_i)$ sont i.i.d, intégrables, d'espérance $R(g)$
- La convergence découle piment de la loi faible des grands nombres :

$$\hat{R}(g, D_n) \xrightarrow{(p)} R(g)$$

En toute généralité, quand F est infinie, c'est beaucoup plus compliqué.

Le principe ERM :

- Au delà de la connaissance du principe ERM ?
- Bornes de généralisation c-a-d, des résultats du type : avec une probabilité supérieure à $1 - \delta$,

$$\forall f \in F, R(f) \leq \hat{R}(f, D_n) + \nu_n$$

- Où ν_n dépend de δ (niveau de confiance), n (échantillon) et de la *complexité* de la classe F de classifieurs considérée. Plus F est complexe, plus il est difficile de garantir un petit ν_n .
- La relation entre δ et la suite ν_n contrôle la *vitesse de convergence*.

Un exemple de notion de complexité : la dimension VC

On dit que F pulvérise l'échantillon D_n si F peut produire tous les étiquetages possibles sur ces n points. Par exemple, la famille des pulvérisateurs linéaires ne fonctionne pas pour 4 points dans le plan.

- On note

$$E(F, D_n) = \{((x_1, f(x_1)), ..., (x_n, f(x_n))), f \in F\}$$

C'est l'ensemble des étiquetages que la famille F peut produire sur les n points.

Soit

$$C(F, n) = \max_{|D_n|=n} |E(F, D_n)|$$

$C(F, n)$ est appelé *fonction de croissance* et mesure le nombre maximum d'étiquetage possibles de n points de l'ensemble X par la classe F. Si

$$C(F, \nu) = 2^\nu$$

, alors on dit que F pulvérise l'échantillon D_n .

- Si F est une classe de fonctions de X dans $\{-1, 1\}$, on définit la dimension VC de F comme :

$$V = \max\{\nu, C(F, \nu) = 2^\nu\}$$

Exemple : pour le classifieur linéaire, la dimension VC est 3. C'est le nombre maximum de points que l'on peut séparer en deux classes par une droite.

Borne de généralisation pour F quelconque ?

On cherche à garantir que ce qu'on voit sur l'échantillon est proche de la réalité avec un terme correctif qui dépend de n. Il y a 3 cas différents :

- Cas $F = \{f_1\}$
- Cas $F = \{f_1, f_2, ..., f_m\}$ fini
- Cas F infini mais de dimension VC finie V

Rappel sur l'inégalité de Hoeffding

Soit $X_1, X_2, ..., X_n$ des variables aléatoires indépendantes telles que $X_i \in [a_i, b_i]$. On note

$$S_n = \sum_{i=1}^n X_i$$

et

$$\mathbb{E}(S_n) = \sum_{i=1}^n \mathbb{E}(X_i)$$

Alors, pour tout $\varepsilon > 0$, on a :

$$\mathbb{P}(\mathbb{E}(S_n) - S_n > \varepsilon) \leq \exp\left(-2 \frac{\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2}\right)$$

et

$$\mathbb{P}(S_n - \mathbb{E}(S_n) > \varepsilon) \leq \exp\left(-2 \frac{\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2}\right)$$

Note : Cette inégalité donne une borne exponentielle sur l'écart entre la moyenne empirique et l'espérance, valable sans hypothèse de variance.

- Cas $F = \{f_1\}$

- Echantillon $D_n = \{(X_i, Y_i)\}_{i=1}^n$
- Perte bornée $L : X \cdot Y \rightarrow [0, 1]$

- pour $f \in F$:
 - $R(f) = \mathbb{E}(f(L(f(x), y))$ et $\hat{R}(f, D_n) = \frac{1}{n} \cdot \sum_{i=1}^n L(f(X_i), Y_i)$

Posons

$$Z_i = \frac{1}{n} \cdot L(f(X_i), Y_i), \text{ alors } Z_i \in \left[0, \frac{1}{n}\right]$$

et

$$S_n = \sum_{i=1}^n Z_i = \hat{R}(f, D_n)$$

d'où

$$E[S_n] = E(Z_1 + Z_2 + ... + Z_n) = R(f)$$

- Par l'inégalité de Hoeffding, pour tout $\varepsilon > 0$, on a :

$$\mathbb{P}(E(S_n) - S_n > \varepsilon) \leq \exp\left(-2 \frac{\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2}\right)$$

Où, ici, $a_i = 0$ et $b_i = \frac{1}{n}$, donc

$$\sum_{i=1}^n (b_i - a_i)^2 = n \cdot \left(\frac{1}{n}\right)^2 = \frac{1}{n}$$

d'où

$$\mathbb{P}\left(R(f_n) - \hat{R}(f_n, D_n) > \varepsilon\right) \leq \exp(-2n\varepsilon^2)$$

Ecrivons

$$\delta = \exp(-2n\varepsilon^2) \Leftrightarrow \varepsilon = \sqrt{\frac{\ln(\frac{1}{\delta})}{2n}}$$

Alors avec une probabilité d'au moins $1 - \delta$, on a :

$$R(f_n) \leq \hat{R}(f_n, D_n) + \sqrt{\frac{\ln(\frac{1}{\delta})}{2n}}$$

car

$$\mathbb{P}\left(R(f) - \hat{R}(f, D_n) > \varepsilon\right) \leq \delta \Leftrightarrow \mathbb{P}\left(R(f) - \hat{R}(f, D_n) \leq \varepsilon\right) \geq 1 - \delta$$

- Cas $F = \{f_1, f_2, ..., f_m\}$ fini

Pour $\varepsilon > 0$ et fixé, on a : $j = 1, ..., p$ définissons l'évènement $A_j(\varepsilon) = \{R(f_j) - \hat{R}(f_j, D_n) > \varepsilon\}$ Alors

$$\sup_{f \in F} \left(R(f) - \hat{R}(f, D_n) > \varepsilon\right) = \cup_{j=1}^p A_j(\varepsilon)$$

Par l'inégalité de Hoeffding, appliquée à chaque f_j , on a :

$$\mathbb{P}(A_j(\varepsilon)) \leq \exp(-2n\varepsilon^2)$$

Par, l'inégalité de borne de l'union :

$$\mathbb{P}\left(\sup_{f \in F} \left(R(f) - \hat{R}(f, D_n) > \varepsilon\right)\right) \leq \sum_{j=1}^p \mathbb{P}(A_j(\varepsilon)) \leq p \cdot \exp(-2n\varepsilon^2)$$

Posons maintenant

$$\delta = p \cdot \exp(-2n\varepsilon^2) \Leftrightarrow \varepsilon = \sqrt{\frac{\ln(\frac{p}{\delta})}{2n}}$$

On obtent donc avec probabilité au moins $1 - \delta$:

$$\forall f \in F, R(f) \leq \hat{R}(f, D_n) + \sqrt{\frac{\ln(\frac{p}{\delta})}{2n}}$$

- Cas F infini mais de dimension VC finie V

Etape 1 : montrer que $\mathbb{P}\left(\sup_{f \in F} |R(f) - \hat{R}(f, D_n)| > \varepsilon\right) \leq 2C(F, 2n) \cdot \exp\left(\frac{-n\varepsilon^2}{\delta}\right)$

Etape 2 : Lemme de Sauer-Shelah

Etape 3 : choix de ε

Séparation train/test

On sépare les données en deux ensembles disjoints :

- *train* (ex. 70 %) pour entraîner le modèle,
 - *test* (ex. 30 %) pour estimer son risque espéré (erreur réelle approximée).
- On évalue donc la performance du modèle sur les données de test, jamais sur celles utilisées pour l'apprentissage.
- Une partie des données est "perdue" pour l'apprentissage (celles mises de côté pour le test).
 - L'estimation du risque espéré peut avoir une variance élevée : le résultat dépend fortement du découpage choisi.

Alternative : la validation croisée
Idée : effectuer plusieurs partitionnements du dataset en train et test.
Exemple : k-fold cross-validation.
• On divise les données en k blocs.
• À chaque itération, on entraîne sur k-1 blocs et on teste sur le bloc restant.
• On répète k fois et on fait la moyenne des erreurs obtenues.

Conclusion

Évaluer un modèle ne se fait pas sur l'erreur d'apprentissage (trop optimiste), mais via une estimation de son risque réel :

- soit par un jeu de test séparé,
- soit, de manière plus robuste, par validation croisée.

Chapitre 2

Classification binaire (perceptron, régression logistique)

Un exemple

On possède des données sur un échantillon de 100 personnes : âge du patient X, présence (1) ou absence (0) d’une maladie cardiaque Y. On cherche un classifieur f de $\mathbb{R}^p$ dans $\{0, 1\}$ permettant de prédire par $y = f(x)$ si un patient X est malade. Ici, une régression linéaire ne convient pas. Il faut une méthode qui intègre la nature binaire de la variable de sortie dans l’expression du classifieur f .

Le perceptron (Rosenblatt, 1958)

On cherche une fonction de prédiction de paramètre

$$w = (b, w) \in \mathbb{R} \cdot \mathbb{R}^p$$

de la forme

$$f_w(x) = \sigma(h_w(x)) \text{ avec } \sigma = \{1 \text{ si } h_w(x) \geq 0, -1 \text{ sinon}\} \text{ et}$$

$$\forall x \in \mathbb{R}^p, h_w(x) = w^T \cdot x + b$$

Ainsi :

$$f_{w(x)} := \begin{cases} 1 & \text{si } w^T \cdot x + b \geq 0 \\ -1 & \text{sinon} \end{cases}$$

On peut réécrire f_w avec $x_i = (1, x_i)$ et $w = (b, w)$ tel que $f(x_i) = \text{sign}(w \cdot x_i)$

Apprentissage des paramètres du perceptron

- On se donne un dataset : $D = \{(x_i, y_i) \in \mathbb{R}^p \cdot \{-1, 1\}\}$
- On veut apprendre les paramètres du perceptron sur ce jeu de données, ce qui revient à trouver le paramètre optimal $w^* = (b^*, w^*)^T$ minimisant la distance entre les exemples mal classés par f_w et la frontière de décision du modèle.

- Un exemple d’indice i est mal classé si $f_w(x_i) = 1$ et $y_i = -1$ ou si $f_w(x_i) = -1$ et $y_i = 1$, c’est à dire si:

$$y_i(w^T \cdot x_i + b) < 0$$

- On veut minimiser la fonction objectif suivante mesurant le risque empirique :

$$\hat{L}(w) = - \sum_{i \in I} y_i(w^T \cdot x_i + b) \text{ avec } I \text{ l'ensemble des indices mal classés}$$

Descente de Gradient Stochastique

Voici une alternative : l’algorithme du perceptron, dans lequel on corrige un exemple mal classés à la fois.

- $k := 0$; Initialiser les poids à zéro $w^0 := (b^0, w^0) = 0$
- Tant qu’il existe $y_i(w^T x_i + b) \leq 0$ (un point mal classé) :
 - Choisir j tel que $y_j(w^T x_j + b) \leq 0$ et $(b^{(k+1)}, w^{(k+1)}) \leftarrow (b^{(k)}, w^{(k)}) + \eta(y_j, y_j x_j)$
 - $k := k + 1$

Note : Dans cette version, on corrige le modèle dans la direction qui ferait bien classer x_j .

En effet,

- Si l’exemple x_j est d’étiquette $y_j = +1$ mais classé comme négatif, alors $w^T x_j + b < 0$. On rajoute x_j aux poids pour orienter l’hyperplan vers ce point.

- Si $y_j = -1$ mais il est classé positif, alors $w^T x_j + b > 0$. On retire x_j des poids (car $y_j = -1$, donc on soustrait).

On peut aussi choisir les indices j du jeu de données de manière aléatoire : c’est une descente de gradient stochastique.

Convergence du perceptron : Théorème de Novikoff, 1962

Ce théorème donne une condition suffisante pour que l’algorithme du perceptron converge en un nombre fini d’itérations.

Supposons que :

- il existe un vecteur $w_{\text{sep}} = (b_{\text{sep}}, w_{\text{sep}})^T \in \mathbb{R} \cdot \mathbb{R}^p$ tel que $\|w_{\text{sep}}\| = 1$, et que

$$\forall i, y_i(w_{\text{sep}}^T \cdot x_i + b_{\text{sep}}) \geq \gamma \geq 0$$

(Ceci est l’hypothèse de de séparabilité linéaire : Il existe un hyperplan séparant les deux classes et chaque point est au moins à une distance gamma de cet hyperplan.)

- $\forall i, \|x_i\| \leq R$ (observations bornées). Alors l’algorithme du perceptron fait au plus

$$\frac{(1 + R)^2}{\gamma^2}$$

itérations avant de converger .

Note : Le premier point implique que les données sont séparables linéairement avec une marge au moins égale à γ . Autrement dit, il existe un hyperplan séparant les deux classes et chaque point est à une distance au moins égale à γ de cet hyperplan. Ainsi, le perceptron converge si les données sont linéairement séparables. Plus les données sont grandes (R), plus la convergence est lente, de meme si la marge de séparation (gamma) est petite (les données sont proches de la frontière).

Preuve : Supposons qu’au bout de k itérations, il existe une observation mal classés (x_j, y_j) , on a alors,

$$w^{(k+1)} \cdot w_{\text{sep}} = w^{(k)} \cdot w_{\text{sep}} + \eta(y_i \cdot b_{\text{sep}} + y_j x_j w_{\text{sep}}) \geq w^{(k)} \cdot w_{\text{sep}} + \eta \cdot \gamma$$

par produit scalaire.

En effet, au k-ieme pas, si on met à jour avec un point mal classé (x_j, y_j) , on a :

$$w^{(k+1)} = w^{(k)} + \eta(y_j, y_j x_j)$$

On en déduit par récurrence que : $w^{(k)} \cdot w_{\text{sep}} \geq k\eta\gamma$.

Comme

$$w^{(k+1)} \cdot w_{\text{sep}} \leq \|w^{(k+1)}\| \|w_{\text{sep}}\| \leq \|w^{(k+1)}\|$$

On a

$$\|w^{(k+1)}\| \geq k\eta\gamma$$

Par ailleurs, l’exemple j est mal classé, donc $y_j(b^{(k)} + w^{(k)} \cdot x_j) \leq 0$ et donc

$$\begin{aligned} \|w^{(k+1)}\|^2 &= \|w^{(k)} + \eta(y_j, y_j x_j)\|^2 \\ &\leq \|w^{(k)}\|^2 + \eta^2 (1 + \|x_j\|^2) + 2\eta y_j (w^{(k)} \cdot (1, x_j)) \\ &\leq \|w^{(k)}\|^2 + \eta^2 (1 + \|x_j\|^2) \end{aligned}$$

car

$$\|(y_j, y_j x_j)\|^2 = y_j^2 (1 + \|x_j\|^2) = 1 + \|x_j\|^2$$

car $y_j \in \{-1, 1\}$

Les x_j étant bornés par R, on a

$$\|w^{(k+1)}\|^2 \leq \|w^{(k)}\|^2 + \eta^2 (1 + R^2)$$

par récurrence,

$$\|w^{(k+1)}\|^2 \leq k\eta^2 (1 + R^2)$$

Donc,

$$k^2 \eta^2 \gamma^2 \leq k\eta^2 (1 + R^2)$$

On en déduit que :

$$k \leq \frac{(1 + R)^2}{\gamma^2}$$

Perceptron non linéaire

On peut chercher un classifieur sous la forme plus générale suivante :

$$f_w(x) = \sigma(h_w(x)) \text{ avec } \forall x \in \mathbb{R}^p, h_w(x) = w^T x + b$$

Où σ est une fonction monotone à valeurs dans $[0, 1]$. C’est la *fonction d’activation*. Ex : La sigmoïde.

Régression logistique

La regression logistique peut etre vue comme une version probabiliste du perceptron. On se place dans le cadre de la *classification binaire* et on va utiliser une approche statistique basée sur la régression logistique.

Le modèle est le suivant :

- Hypothèse : Il y a une distribution conditionnelle de Y (loi de Bernoulli de paramètre p(x) qui dépend de la valeur x de X).

$$\mathbb{P}(Y = 1 \mid X = x) = p(x) \text{ et } \mathbb{P}(Y = 0 \mid X = x) = 1 - p(x)$$

Ainsi,

$$\mathbb{P}(Y = y \mid X = x) = p(x)^y (1 - p(x))^{1-y}$$

Seulement, $p(x)$ doit vérifier des contraintes, que sont :

- Valeurs dans l’intervalle $[0, 1]$ (car c’est une probabilité)
- Monotonie en x
- Stabilité par changement d’origine et d’échelle sur la variable explicative :

si $p \in L$, alors $x \mapsto p(a_0 + a_1 x)$ aussi.

Le cas général :

$$p(x) = \sigma(w^T x + b)$$

avec

$$w' = (w, b)^T \in \mathbb{R}^p \cdot \mathbb{R}$$

et σ est une fonction non linéaire.

Le modèle Logit est :

$$\sigma(t) = \frac{\exp(t)}{1 + \exp(t)} = \frac{1}{1 + \exp(-t)}$$

Donc

$$\mathbb{P}(Y = 1 \mid X = x) = \frac{1}{1 + \exp(-(w^T x + b))}$$

Il y a 2 paramètres à estimer : b et w .

Estimation par Maximum de Vraisemblance (MV)

- Hypothèse : v.a. Y_i indépendantes et de lois de Bernouilli $p_{w(x_i)}$.
- La vraisemblance associée à la suite d’observations $y = (y_1, ..., y_n)$ pour la suite de valeurs explicatives $x = (x_1, ..., x_n)$ est :

$$L_n(y \mid x, w) = \prod_{i=1}^n p_w(x_i)^{y_i} (1 - p_w(x_i))^{1-y_i}$$

La log-vraisemblance est donc :

$$l_n(y \mid x, w) = \sum_{i=1}^n ([y_i \log(p_w(x_i)) + (1 - y_i) \log(1 - p_w(x_i))])$$

Entropie : Rappel

En théorie de l’information, l’entropie d’une loi p est :

$$H(p) = - \sum_{k=1} p_k \log(p_k)$$

L’entropie croisée entre deux lois p (la vraie) et q (le modèle) est :

$$H(p, q) = - \sum_{k=1} p_k \log(q_k)$$

Regression logistique :

$$y_i \in \{0, 1\}$$

La loi vraie p : $P(y_i = 1) = y_i, P(y_i = 0) = 1 - y_i$

La loi du modèle : $Q(\hat{y}_i = 1) = \hat{p}_i, Q(\hat{y}_i = 0) = 1 - \hat{p}_i$

L’entropie croisée est : $H(p, q) = -[y_i \cdot \log(\hat{p}_i) + (1 - y_i) \cdot \log(1 - \hat{p}_i)]$

On remarque que c’est l’opposé de la log-vraisemblance. Ainsi, maximiser la log-vraisemblance revient à minimiser l’entropie croisée.

Note : La cross-entropy loss traduit la même idée que le maximum de vraisemblance : elle mesure à quel point les prédictions d’un modèle correspondent aux données réelles. C’est pourquoi, en machine learning, on l’utilise comme fonction de coût de référence pour entraîner les modèles de classification.

Estimation par maximum de vraisemblance

En partant de la log-vraisemblance, on obtient les équations suivantes :

$$\frac{\partial l_n(y \mid x, w)}{\partial b} = \sum_i \left(y_i - \frac{\exp(b + w^T x_i)}{1 + \exp(b + w^T x_i)} \right) = 0$$

et

$$\frac{\partial l_n(y \mid x, w)}{\partial w} = \sum_i x_i \left(y_i - \frac{\exp(b + w^T x_i)}{1 + \exp(b + w^T x_i)} \right) = 0$$

Pour les résoudre, on peut utiliser une méthode de type Newton-Raphson (descente de gradient).

Théorème de convergence asymptotique de l’EMV (Estimateur du Maximum de Vraisemblance)

Hypothèse : les x_i sont i.i.d de loi de densité λ non nulle sur un compact K.

Alors la log-vraisemblance l_n est strictement concave.

De plus, si la matrice de Fisher

$$I = \int_K x^t x p(x) (1 - p(x)) \lambda(x) dx$$

est inversible et que l’espace des paramètres est compact alors :

$$\sqrt{n}(w^{(n)} - w) \xrightarrow{L} N(0, I^{-1})$$

Conclusion : Sous les conditions du compact, de la convexité et de l’inversibilité de la matrice de Fisher, l’estimateur du maximum de emblance converge vers le vrai paramètre w^* , avec une variance donnée par I^{-1} .

Note : Ce théorème garantit que le modèle de classification logistique est statistiquement consistant, car avec beaucoup de données, le modèle apprend les bons paramètres.

Preuve :

On a $l_n(w) = \sum_{i=1}^n L$ et

$$\nabla(l_n)(w) = \sum_{i=1}^n (y_i - p_w(x_i)) x_i$$

d’où :

$$\nabla^2(l_n)(w) = - \sum_{i=1}^n p_w(x_i) (1 - p_w(x_i)) x_i x_i^t$$

La matrice d’information de Fisher est :

$$\begin{aligned} I &= \mathbb{E}(p_w(X)(1 - p_w(X))XX^t) \\ &= \int_K x x^T p_w(x) (1 - p_w(x)) \lambda(x) dx \end{aligned}$$

or, on a

$$\begin{aligned} p_w(x)(1 - p_w(x)) &= -p_w(x)^2 + p_w(x) \\ &= -\left(p_w(x)^2 - p_w(x) + \frac{1}{4}\right) + \frac{1}{4} \\ &= \left(-p_w(x) + \frac{1}{2}\right)^2 + \frac{1}{4} \end{aligned}$$

Pour tout $w_i, p_w(x)(1 - p_w(x)) \in [0, \frac{1}{4}]$ et $H_i(w) = \nabla^2(l_n)(w)$ est définie négative donc l_n est strictement concave. Donc l’EMV w^* existe et est unique, défini par $\nabla l_n(w^*) = 0$

Appliquons Taylor à $\nabla l_n(w)$ au point w^* .

$0 = \nabla l_n(w^*) = \nabla l_n(w) + H_n(w_n)(w_n - w^*)$ où w_n est sur le segment entre w et w^* .

On isole : $\sqrt{n}(w_n - w^*) = \left[-\frac{1}{n}H_n(w_n)\right]^{-1} \cdot \frac{1}{\sqrt{n}}\nabla(l_n(w))$

Par le TLC (Le Théorème Central Limite) :

$$\frac{1}{\sqrt{n}}\nabla(l_n(w)) = \frac{1}{\sqrt{n}}\sum_{i=1}^n (y_i - p_w(x_i))x_i \xrightarrow{L} N(0, I)$$

Par la loi des grands nombres,

$$-\frac{1}{n}H_{n(w_n)} \xrightarrow{P} I.$$

Ainsi, on a bien la convergence en loi :

$$\sqrt{n}(w_n - w^*) \xrightarrow{L} N(0, I^{-1})$$

L’interprétation de ce résultat est :

- la consistance de l’EMV(Estimateur du Maximum de Vraisemblance) :

$$w^* \xrightarrow{P} w$$

- $\hat{w}^{(n)}$ est asymptotiquement normal : l’erreur d’estimation rééchelonnée par $\sqrt{n}$ suit une loi normale centrée.

Cela permet de faire de l’inférence statistique (tests, intervalles de confiance).

Ce qu’il faut retenir du chapitre :

- Le perceptron est un algorithme de classification binaire linéaire qui fonctionne seulement si les données sont linéairement séparables et qui converge en un nombre fini d’itérations dans ce cas.
- La régression logistique est toujours définie et possède des propriétés statistiques fortes, ce qui en fait un classifieur fiable en pratique.

Chapitre 3

SVM linéaires

Un SVM (Support Vector Machine), aussi appelé modèle des *séparateurs à vaste marge*.

Un problème de classification binaire est dit linéairement séparable s'il existe un hyperplan séparant deux classes.

Définitions: Marge : La marge d'un classifieur linéaire est la distance entre l'hyperplan et les points d'apprentissage les plus proches (appelés vecteurs de supports).

Base d'apprentissage : $\{(x_i, y_i), i = 1, ..., n\}, x_i \in \mathbb{R}^d, y_i \in \{-1, 1\}$

Soit $h_w(x) = w^T x + b$

- $h(x) = 0$: hyperplan (surface de réparation)
- $h(x) > 0$: classe 1 ($y_i = 1$)
- $h(x) < 0$: classe 2 ($y_i = -1$)

La fonction de décision est $f_w(x) = \text{signe}(h_w(x))$. w est le vecteur normal à l'hyperplan et b est le biais, qui positionne l'hyperplan par rapport à l'origine. (w, b) et (kw, kb) définissent exactement la même surface de séparation pour tout $k > 0$.

Formalisation du problème

Si v est un vecteur support et si H est l'hyperplan séparateur $H = \{x, w^T x + b = 0\}$ alors la marge est :

$$2d(v, H) = \frac{2 |w^T v + b|}{\|w\|}$$

On impose la condition de normalisation $|w^T x + b| = 1$ pour les vecteurs de support :

$$\text{marge} = \frac{2}{\|w\|}$$

On cherche donc à maximiser la marge, ce qui revient à minimiser $\|w\|$. Optimisation de la marge :

$$\begin{cases} \text{Trouver } w \\ \min \|w\|^2 \\ \text{t.q } (\forall i, 1 - y_i(w^T x_i + b) \leq 0) \end{cases}$$

Il s'agit d'un problème d'optimisation sous contraintes inégalités. Grâce à la théorie de l'optimisation convexe (Lagrangien), ce problème peut se ramener à un problème dual, qui est :

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i^T \cdot x_j)$$

t.q pour tout $i, \alpha_i \geq 0$ et $\sum_i \alpha_i y_i = 0$

Note : La théorie concernant l'optimisation convexe sous contraintes nous dis que pour minimiser $f(w) = \|w\|^2$, on peut commencer par résoudre le problème dual.

Démonstration : on suppose les données séparable linéairement.

$$(P) \min_{w,b} \frac{1}{2} \|w\|^2$$

$$\text{s.c } 1 - y_i(w^T x_i + b) \leq 0$$

On introduit un multiplicateur de Lagrange $\alpha_i \geq 0$ pour chaque contrainte, et on forme :

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n (\alpha_i)(y_i(w^T x_i + b) - 1)$$

La fonction duale est :

$$g(\alpha) = \inf_{w,b} L(w, b, \alpha)$$

Calcul de la borne inf:

$$\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i x_i$$

Cela signifie que le vecteur optimal w est une combinaison linéaire des données d'entraînement.

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^n \alpha_i y_i = 0$$

En remplaçant w dans L , en imposant $\sum \alpha_i y_i = 0$, on obtient :

$$g(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T \cdot x_j)$$

Le problème dual est donc : (D) :

$$\max_{\alpha \in \mathbb{R}^2} \sum \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j$$

s.c

$$\alpha_i \geq 0 \text{ et } \sum_{i=1}^n \alpha_i y_i = 0$$

Par le théorème de Slater, (P) et (D) sont équivalents.

Algorithme SVM marge dure

On prends en entrée une base d'apprentissage $\{(x_i, y_i), i = 1, ..., n\}, x_i \in \mathbb{R}^d$. et $y_i \in \{-1, 1\}$

et on cherche à résoudre le problème dual (D). C'est à dire qu'on cherche une solution α^*

solution de

$$\max_{\alpha \in \mathbb{R}^n} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i^T \cdot x_j)$$

sous les contraintes de $\alpha_i \geq 0$ et $\sum_i \alpha_i y_i = 0$.

Une fois qu'on a trouvé la solution optimale $\alpha^* = (\alpha_1^*, ..., \alpha_n^*)$, on en déduit le vecteur des poids :

$$w^* = \sum_{i=1}^n \alpha_i^* y_i x_i$$

et le biais $b^* = y_i - \sum_{j=1}^n \alpha_j^* y_j (x_j^T \cdot x_i)$ pour un i tel que $\alpha_i^* > 0$ (vecteur de support).

La fonction de décision est alors :

$$f(x) = \text{signe} \left(\sum_{i=1}^n \alpha_i^* y_i x_i^T \cdot x + b^* \right)$$

, elle est basé uniquement sur les vecteurs de support (ceux pour lesquels $\alpha_i^* > 0$).

Cas non séparable

Dans le cas où les données ne sont pas séparables linéairement, on introduit une technique dite de *marge souple* (*soft margin*) qui consiste à autoriser certaines erreurs de

classification. Ces contraintes sont modélisées par des variables d'écart $\xi_i \geq 0$. Si le point est correctement classé avec marge :

$$y_i(w^T x_i + b) \geq 1 \Rightarrow \xi_i = 0$$

Si le point est mal classé :

$$y_i(w^T x_i + b) < 1 \Rightarrow \xi_i = 1 - y_i(w^T x_i + b) > 0$$

Ainsi $\xi_i = \max(1 - y_i(w^T x_i + b), 0)$.

L'objectif est de minimiser la norme de w (rappel):

$$\begin{cases} \text{Trouver } w \\ \min \|w\|^2 \\ \text{t.q } (\forall i, 1 - y_i(w^T x_i + b) \leq 0) \end{cases}$$

Si toutes les variables d'écart sont nulles, on retrouve le problème séparable linéairement.

Puisque il faut minimiser les deux termes simultanément on introduit une variable

d'équilibrage $C > 0$ qui permet d'avoir une seule fonction objectif dans le problème

d'optimisation $\min \|w\|^2 + C \sum_{\{i\}} \xi_i$.

Algorithme SVM marge souple

On prends en entrée une base d'apprentissage $\{(x_i, y_i), i = 1, ..., n\}, x_i \in \mathbb{R}^d$. et $y_i \in \{-1, 1\}$

et on cherche à résoudre le problème dual :

$$\max_{\alpha \in \mathbb{R}^n} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i^T \cdot x_j)$$

s.c

$$0 \leq \alpha_i \leq C \text{ et } \sum_i \alpha_i y_i = 0$$

Pour cela, on choisit $i \in \{1, ..., n\}$ tel que $0 < \alpha_i^* < C$ et on calcule le biais :

$$b^* = y_i - \sum_{j=1}^n \alpha_j^* y_j (x_j^T \cdot x_i)$$

et le vecteur des poids :

$$w^* = \sum_{i=1}^n \alpha_i^* y_i x_i$$

.

Ainsi, la fonction de décision est :

$$f(x) = \text{signe} \left(\sum_{i=1}^n \alpha_i^* y_i x_i^T \cdot x + b^* \right)$$

, elle est basé uniquement sur les vecteurs de support (ceux pour lesquels $\alpha_i^* > 0$).

SVM non linéaires

Pour rappel, la démarche concernant les SVM linéaires est la suivante :

1. On reformule le problème d'optimisation dans le dual.
2. On trouve la valeur optimale des variables duales α .
3. On en déduit le vecteur des poids w : $w^* = \sum_{i=1}^n \alpha_i^* y_i x_i$.
4. Cette expression permet une fois qu'on a trouvé w^* de calculer h_{w^*} pour une nouvelle observation x :

$$\sum_{i \in A} \alpha_i y_i (x_i^T \cdot x)$$

.

Le point clé est de disposer d'un produit scalaire. C'est ce qu'on va généraliser aux SVM non linéaires à l'aide de la technique du *noyau*.

Technique du noyau

Le noyau K est une mesure de similarité entre deux points. C'est une fonction qui prend en entrée deux vecteurs et qui retourne un scalaire :

- $x, y \in X, K(x, y) \geq 0, K(x, y) = K(y, x)$

- Si $x \neq y$, alors $K(x, y) > K(x, x)$

- $x = y$ ssi $K(x, y) = K(x, x)$

Définition : Un espace de Hilbert est un espace vectoriel H muni d'un produit scalaire $\langle \cdot, \cdot \rangle$ tel que H est complet pour la norme associée $\|\cdot\| = \sqrt{\langle \cdot, \cdot \rangle}$.

Théorème de Mercer

Soit X , un compact de $\mathbb{R}^d$ et $K : X \times X \mapsto \mathbb{R}$ une fonction symétrique ($K(x, y) = K(y, x)$). Supposons de plus que, $\forall f \in L^2(X)$:

$$\int K(x, y) f(x) f(y) dx \geq 0 \text{ (Condition de Mercer)}$$

Alors il existe un espace de Hilbert H et une application $\varphi : X \mapsto H$ tel que,

$$\forall x, y \in X, K(x, y) = \langle \varphi(x), \varphi(y) \rangle \text{ (produit scalaire).}$$

Ce théorème fait le lien entre le noyau et le produit scalaire.

Condition équivalente : Noyau positif défini

Pour tout $n \in \mathbb{N}$ et $\{x_i\}$ la matrice de Gramm

$$K = [K(x_i, x_j)]_{i,j}$$

est définie positive, et

$$\forall c \in \mathbb{R}^n, c \neq 0, \text{ on a } c^T K c > 0.$$

Les SVM sont des séparateurs linéaires dont l'avantage est un problème d'optimisation convexe. La question est de savoir comment étendre ce résultat aux données non linéairement séparables.

Le Kernel trick

Le principe est de transposer les données dans un autre espace (en général de plus grande dimension) dans lequel elles sont linéairement séparables ou presque et ensuite appliquer l'algorithme SVM sur les données transposées. La transformation est :

$$\varphi : \mathbb{R}^d \mapsto H$$

, où H est un espace de Hilbert. Si K est un noyau défini positif, alors l'existence de φ est garantie par le théorème de Mercer.

Note : On ne calcule jamais explicitement $\varphi(x)$, mais uniquement les produits scalaires $\langle \varphi(x_i), \varphi(x_j) \rangle = K(x_i, x_j)$.

Le problème d'optimisation pour les SVM non linéaires est donc :

$$\begin{cases} \text{Trouver } W_H = \varphi(w) \\ \min \|W_H\|^2 \\ \text{t.q } (\forall i, 1 - y_i(W_H^T \varphi(x_i) + b) \leq 0) \end{cases}$$

et le problème dual est :

$$\max_{\alpha \in \mathbb{R}^n} \sum_i \alpha_i - \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

s.c

$$0 \leq \alpha_i \leq C \text{ et } \sum_i \alpha_i y_i = 0$$

Ainsi, la fonction de décision est :

$$f(x) = \sum_{i=1}^n \alpha_i^* y_i K(x_i, x)$$

avec les α^* issus du problème dual:

$$f^*(x) = \sum_{i=1}^n \alpha_i^* y_i K(x_i, x) + b_i^*$$

avec

$$b^* = \frac{1}{I} \sum_{x_j \in I} \left(y_j - \sum_{i=1}^n \alpha_i^* y_i K(x_i, x_j) \right)$$

et I est l'ensemble des vecteurs de support tels que $0 < \alpha_j^* < \hat{a}$ (paramètre du noyau).

Chapitre 4

Neurones biologiques et neurones formels

Le système nerveux humain est composé de neurones biologiques qui communiquent entre eux via des synapses. Chaque neurone reçoit des signaux d'autres neurones, les traite, puis transmet un signal de sortie à d'autres neurones. Le système nerveux humain est composé d'environ 100 milliards de neurones et 10 000 synapses par neurone. On peut modéliser un neurone biologique par un neurone formel, qui est une unité de base dans les réseaux de neurones artificiels. Un neurone formel reçoit des entrées pondérées, applique une fonction d'activation, et produit une sortie.

Fonction d'activation

La fonction d'activation du neurone s'appliquera sur le résultat du produit scalaire entre les entrées du neurone et les poids synaptiques correspondants, auquel on ajoute w_{i0} . Il existe plusieurs fonctions d'activation, parmi lesquelles :

- La fonction sigmoïde : $f(x) = \frac{1}{1+e^{-x}}$
- La fonction tanh : $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- La fonction ReLU (Rectified Linear Unit) : $f(x) = \max(0, x)$
- La fonction softmax : $f(x_i) = \frac{e^{x_i}}{\sum_j (e^{x_j})}$ pour chaque x_i dans le vecteur d'entrée.

Le perceptron

L'objectif du perceptron est de classer des données linéairement séparables. Pour cela, il trouve une surface séparatrice entre les différentes classes. Formellement : On dispose d'un hyperplan dans $\mathbb{R}^n$ défini par l'équation

$$\sum_i w_i x_i + w_0 = 0$$

. x est dans la classe -1 si

$$\sum_i w_i x_i + w_0 < 0$$

et dans la classe $+1$ sinon. On utilise la fonction seuil, définie par $f(x) = 1$ si $x \geq 0$ et $f(x) = -1$ sinon.

Les réseaux multicouches

Dans un réseau de neurones multicouches, les neurones d'entrée n'ont pas de fonctions d'activation : leurs états étant imposés de l'extérieur. Les neurones des couches cachées ont des fonctions d'activation sigmoïdes. Les neurones de sorties, suivant les applications, ont des fonctions d'activation sigmoïdes, linéaires, softmax.

Souvent, les connexions sont complètes entre couches. Elles sont orientées d'une couche i vers une couche j supérieure à i . Il n'y a donc pas de connexion d'une cellule vers une cellule de niveau inférieur (donc pas de boucle), ni de connexion entre cellules d'une même couche. Tous les neurones, sauf ceux d'entrée, sont également affectés d'une connexion de seuil. Ce qui signifie qu'ils reçoivent une entrée constante égale à 1, avec un poids associé.

Propagation avant des états

La propagation avant des états dans un réseau de neurones multicouches consiste à calculer les sorties de chaque couche en utilisant les entrées de la couche précédente et les poids synaptiques associés.

On présente un vecteur input x à la couche d'entrée et il sera propagé d'une couche à une autre vers la couche de sortie.

y : étant le vecteur de sortie "output" calculé.
 G : fonction définie par le réseau : $y = G(x, W)$.
 W représente l'ensemble des poids synaptiques et des seuils.

Apprentissage

On dispose d'un ensemble d'apprentissage $\{(x^{(k)}, y^{(k)}), k = 1, ..., N\}$ de N exemples. Chaque exemple est constitué d'une entrée $x^{(k)}$ et d'une sortie désirée $y^{(k)}$.

Le reseau définit une fonction

$$Y = G(X, W).$$

Pour un individu, le réseau prédit la sortie

$$y^{(k)}.$$

L'apprentissage consiste à trouver les poids W tel que pour tout $x^{(k)}, \widehat{y^{(k)}} \sim y^{(k)}$, où $\widehat{y^{(k)}}$ est la sortie prédite par le réseau pour l'entrée $x^{(k)}$. C'est à dire minimiser une fonction de coût (ou fonction de perte) qui mesure l'écart entre les sorties prédites par le réseau et les sorties désirées.

Formellement, l'erreur entre $\widehat{y^{(k)}}$ et $y^{(k)}$ est définie par la fonction de coût quadratique :

$$E(W) = \sum_{k=1}^N \| \widehat{y^{(k)}} - y^{(k)} \|^2$$

L'apprentissage consiste donc à minimiser $E(W)$ par rapport aux poids W . Pour cela, on utilise la méthode du gradient.

Algorithme de rétropropagation

L'algorithme de rétropropagation est une méthode efficace pour calculer les gradients de la fonction de coût par rapport aux poids du réseau. Il utilise la règle de la chaîne pour propager l'erreur depuis la couche de sortie vers les couches précédentes.

On calcule la dérivée partielle de l'erreur par rapport aux poids en utilisant la règle de la chaîne :

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial a_j} \cdot \frac{\partial a_j}{\partial w_{ij}} = z_i s_j.$$

où a_j est l'activation du neurone j , z_i est la sortie du neurone i , et s_j est le signal d'erreur pour le neurone j .

Les réseaux convolutionnels

Les *réseaux de neurones convolutionnels (CNN)* sont particulièrement adaptés pour le traitement des données structurées en grille, comme les images. Ils utilisent des couches de convolution pour extraire des caractéristiques locales des données d'entrée. Au perceptron multicouches on va rajouter d'autres couches préliminaires-les couches convolutionnelles qui vont permettre de tenir compte de la nature des entrees (images, vidéos etc...) et d'avoir des propriétés d'invariance par translation du réseau.

On a en entree une image I de taille (H, W) (Height, Width). Dans le cas d'une image noir et blanc, il s'agit d'une matrice $I(p_1, p_2)$ qui a chaque pixel (p_1, p_2) associe une couleur a valeurs dans $\{0, \dots, 255\}$. Dans le cas d'une image couleur, il s'agit d'un tenseur $I(p_1, p_2, c)$ qui a chaque pixel (p_1, p_2) et a chaque channel c associe une couleur a valeurs dans $\{0, \dots, 255\}$.

Afin que la tache réalisée par le réseau soit robuste aux changements d'orientation des images, on peut de maniere préliminaire faire de la *data augmentation* et translater, symmetriser et faire subir des rotations aux images du jeu de données.

Par définition, Un *filtre convolutionnel* est une transformation lineaire appliquée à une image. Le but d'une couche convolutionnelle est d'appliquer un ou plusieurs filtres a l'image afin de *représenter les données de maniere pertinente par un vecteur*.

La sortie du bloc est une matrice. On veut alimenter un classifieur qui prend comme entree un vecteur. Il faut donc mettre a plat la sortie précédente en utilisant une couche de type Flatten. :

```
from keras.models import Sequential
from keras.layers import Conv2D, Flatten
model.add(Flatten())
```

Definition : Dropout Afin d'éviter le surapprentissage (trop de paramètres), au lieu d'apprendre tous les paramètres d'un réseau de neurones, on en apprend une fraction. Pour cela, on utilise la technique du *dropout*, qui consiste à désactiver aléatoirement un pourcentage de neurones pendant l'entraînement. Cela permet de réduire la complexité du modèle et d'améliorer sa capacité de généralisation.

Definition : le transfert learning Le transfert learning est une technique populaire en ML car elle permet de gagner du temps. L'idée, au lieu de construire soi même son réseau de neurones et d'apprendre les poids, est d'utiliser des réseaux pré-entraînés. L'approche usuelle consiste à récupérer les couches convolutionnelles d'un réseau déjà utilisé dans une tâche proche de la nôtre et à apprendre uniquement la partie perceptron multicouches.

Séries temporelles et réseaux de neurones récurrents

L'objectif des RNN est de prédire les valeurs d'une série temporelle en ayant connaissance de son historique. cela va nécessiter l'introduction d'architecture spécifiques.

Note : Les RNN sont sujets au problème du vanishing gradient, ce qui amène à l'apparition de LSTM (Long Short Term Memory) et GRU (Gated Recurrent Units). Pour rappel, le vanishing gradient est un problème qui survient lors de l'entraînement des réseaux de neurones profonds, où les gradients des couches profondes deviennent extrêmement petits, rendant l'apprentissage inefficace.

Rappels d'optimisation

Chaque réseau de neurones nécessite un *optimizer*. Un exemple :

```
from keras import optimizers

model = Sequential()
model.add(Dense(64, kernel_initializer='uniform', input_shape=(10,)))
model.add(Activation('softmax'))
sgd = optimizers.SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
model.compile(loss='mean_squared_error', optimizer=sgd)
```

La descente de gradient

La descente de gradient est une méthode d'optimisation itérative utilisée pour minimiser une fonction de coût en ajustant les paramètres du modèle dans la direction opposée au gradient de la fonction de coût par rapport à ces paramètres. Initialement : θ^0

On met à jour les paramètres selon la règle :

Pour tout

$$t, \theta^{(t+1)} \leftarrow \theta^t - \eta \nabla E(\theta^t)$$

où η est le learning rate.

Pour *rappel*, les principaux resultats mathématiques sont valables pour des fonctions convexes i.e. satisfaisant

$$\forall \lambda \in [0, 1], E(\lambda \theta_1 + (1 - \lambda) \theta_2) \leq \lambda E(\theta_1) + (1 - \lambda) E(\theta_2)$$

Concernant le taux d'apprentissage η :

- Si on prend pour η une valeur trop grande, alors on a un apprentissage trop rapide et incapable de converger vers un minima local.
- Si on prend pour η une valeur trop petite, alors on a un apprentissage trop lent et la convergence vers un bon minima local sera trop longue.

Stochastic Gradient Descent (SGD)

Supposons que nous ayons 10,000 observations, 10 features et que la sortie est réelle i.e.

$$D = \{(x_i, y_i), i = 1, ..., 10000\}$$

avec pour tout

$$i, x_i \in \mathbb{R}^{10}, y_i \in \mathbb{R}.$$

La somme consiste d'autant de termes que d'observations dans le jeu de données, 10000 termes dans notre cas. Alors, la fonction objectif $E(\theta)$ peut être par exemple la somme des résidus au carré :

$$E(\theta) = \sum_{i=1}^{10000} E_{i(\theta)}$$

avec pour tout

$$i, E_{i(\theta)} = \|y_i - f_{\theta}(x_i)\|^2$$

On doit calculer la dérivée par rapport à chaque feature. Il y a donc 10 features, soit 10 poids à mettre à jour. Pour chaque poids, on doit calculer la somme de 10000 termes. Le calcul du gradient demande donc $10000 \times 10 = 100\,000$ opérations par itération. On prend en général 1000 itérations. On a alors $100\,000 \times 1000 = 100\,000\,000$ opérations pour finaliser l'algorithme. Sur cet exemple on voit que la descente de gradient prend beaucoup de temps quand on a beaucoup de données.

L'idée de la Stochastic Gradient Descent (SGD) est de ne pas calculer le gradient sur l'ensemble des données, mais sur un seul exemple choisi aléatoirement à chaque itération. Ainsi, à chaque itération, on met à jour les paramètres en utilisant uniquement l'exemple (x_i, y_i) choisi aléatoirement :

$$\theta^{(t+1)} \leftarrow \theta^t - \eta \nabla E_{i(\theta^t)}$$

Théorème : Supposons que la fonction E soit convexe et pour tout t ,

$$\sup \mathbb{E} \left[\nabla E_{i(\theta^{(t)})} \right] < \infty$$

. Soit η_t une suite à terme positifs tels que

$$\sum \eta_t = \infty \text{ et } \sum \eta_t^2 < \infty.$$

Alors l'algorithme du gradient stochastique converge presque sûrement vers le minima de E .

SGD avec mini-batch

Pour diminuer la variance de l'estimateur, on peut remplacer le gradient par la moyenne sur un mini batch de taille 16, 32 ou 64 observations. Il y a un compromis entre la taille du mini-batch et le biais de l'estimateur. On met à jour les paramètres selon la règle :

Pour tout

$$t, \theta^{(t+1)} \leftarrow \theta^t - \eta \left(\frac{1}{m} \right) \sum_{i=1}^m \nabla E_{b_i}(\theta^t)$$

où

$$\{b_1, ..., b_m\}$$

est un mini-batch d'observations choisi aléatoirement.

SGD avec momentum

On peut améliorer la méthode SGD en utilisant le momentum, couplé avec le SGD. Cela permet d'accélérer la convergence en rajoutant un terme additionnel :

$$\nu_t = \gamma \nu_{t-1} + \eta \times \text{grad}$$

et

$$\theta \leftarrow \theta - \nu_t = \theta - \gamma \nu_{t-1} - \eta \times \text{grad}$$

Pour faire simple, le momentum permet de lisser les mises à jour des paramètres en prenant en compte les mises à jour précédentes. Cela aide à éviter les oscillations et à accélérer la convergence vers le minimum.

SGD avec RMSprop

RMSprop (Root Mean Square Propagation) est une méthode d'optimisation qui adapte le taux d'apprentissage pour chaque paramètre en fonction de la moyenne des carrés des gradients récents. Cela permet de stabiliser les mises à jour des paramètres et d'améliorer la convergence.

Methode ADAM

Adam (Adaptive Moment Estimation) est une méthode d'optimisation qui combine les avantages de RMSprop et du momentum. Il calcule des moyennes mobiles des gradients et des carrés des gradients pour adapter le taux d'apprentissage pour chaque paramètre.